

Micro UART Project Report

Micro UART

Author Name: Krishna Girish

Project Introduction

The project implements a parameterized Micro UART (Universal Asynchronous Receiver Transmitter) using Verilog HDL. The UART design supports serial communication between transmitter and receiver modules using asynchronous communication methodology. The design consists of UART transmitter, UART receiver, baud rate generator, and oversampling clock generator modules integrated together through a top-level module.

The UART transmitter converts parallel input data into serial data format by appending start and stop bits. The UART receiver performs serial-to-parallel conversion using oversampling and center sampling techniques for reliable data reception. The receiver operates at 16x oversampling frequency to accurately sample incoming serial data at the center of each bit period.

The design supports variable data widths from 6-bit to 8-bit transmission. The UART frame consists of:

- One start bit
- 6 to 8 data bits
- One stop bit
- No parity bit

The transmitter and receiver are synchronized using baud rate clock generation modules. The baud generator produces the transmission baud clock while an additional 16x baud clock generator is implemented for receiver oversampling functionality.

The UART transmitter operates using a finite state machine (FSM) consisting of IDLE, START, DATA, and STOP states. The receiver also uses FSM-based implementation along with oversampling logic to detect the start bit, sample incoming data bits at the center, and reconstruct the received parallel data.

The verification environment includes driver, scoreboard, reference model, and top-level testbench architecture. The testbench validates proper UART transmission and reception functionality for multiple input patterns. A reference model is used to independently generate expected UART behavior and compare DUT outputs.

The project was simulated using Vivado for RTL functional verification and QuestaSim for waveform analysis and coverage generation.

Objectives

- To design and implement a parameterized UART transmitter and receiver using Verilog HDL.
 - To implement asynchronous serial communication using start and stop bits.
 - To support configurable data widths from 6-bit to 8-bit.
 - To implement baud rate generation for UART communication.
 - To implement 16x oversampling in the UART receiver.
 - To perform center sampling for accurate data reception.
 - To design FSM-based transmitter and receiver architectures.
 - To implement serial-to-parallel and parallel-to-serial data conversion.
 - To verify UART functionality using self-checking testbench architecture.
 - To validate UART behavior under multiple data transmission conditions.
 - To perform simulation and waveform analysis.
 - To perform code coverage analysis using QuestaSim.
-

Design Architecture

The Micro UART design consists of the following major modules:

- Top UART Module
- Baud Rate Generator
- 16x Baud Generator
- UART Transmitter
- UART Receiver

The top-level UART module integrates both transmitter and receiver modules along with the baud generation circuitry.

Inputs

SIGNAL	WIDTH	DESCRIPTION
sys_clk	1 bit	System clock input
sys_rst_l	1 bit	Active low asynchronous reset
xmitH	1 bit	Transmission enable signal

SIGNAL	WIDTH	DESCRIPTION
xmit_dataH	Parameterized	Parallel transmit data input

Outputs

SIGNAL	WIDTH	DESCRIPTION
uart_XMIT_dataH	1 bit	Serial UART transmit line
xmit_doneH	1 bit	Transmission complete indication
xmit_active	1 bit	Indicates transmission in progress
rec_dataH	Parameterized	Received parallel data
rec_readyH	1 bit	Indicates valid received data
rec_busy	1 bit	Indicates receiver busy state

UART Frame Format

The UART communication frame implemented in this project consists of:

FIELD	DESCRIPTION
Start Bit	Logic 0 indicating beginning of transmission
Data Bits	6-bit to 8-bit configurable data
Stop Bit	Logic 1 indicating end of transmission

Parity functionality is not implemented in this design.

Baud Rate Generator

The baud generator module generates the transmission clock used by the UART transmitter. A counter-based divider circuit is implemented to divide the system clock into the required baud frequency.

The module toggles the baud enable signal after the configured divider count reaches the programmed value.

The receiver uses a separate 16x baud generator module which produces an oversampling clock operating at sixteen times the transmitter baud rate.

The oversampling clock improves receiver accuracy and enables center sampling functionality.

UART Transmitter Architecture

The UART transmitter converts parallel input data into serial data format using a finite state machine.

Transmitter States

STATE	DESCRIPTION
IDLE	Waits for transmission request
START	Transmits start bit
DATA	Transmits serial data bits
STOP	Transmits stop bit

During transmission, input data is loaded into an internal shift register. Data bits are transmitted serially starting from the least significant bit (LSB).

The transmitter generates `xmit_active` signal while transmission is in progress and asserts `xmit_doneH` after successful completion of the stop bit.

UART Receiver Architecture

The UART receiver reconstructs parallel data from the incoming serial UART stream.

The receiver uses:

- 16x oversampling
- Center sampling
- Double synchronization for metastability reduction
- FSM-based reception control

Receiver States

STATE	DESCRIPTION
IDLE	Waits for start bit detection
START	Validates start bit
DATA	Receives serial data bits
STOP	Validates stop bit

The receiver samples incoming data at the center of the bit period using `sample_count` logic. The start bit is validated at sample count 7 to ensure accurate synchronization.

During data reception, serial bits are shifted into an internal shift register and reconstructed into parallel data.

After successful stop bit detection:

- `rec_dataH` stores received data
 - `rec_readyH` is asserted
 - `rec_busy` is deasserted
-

Oversampling and Center Sampling

The UART receiver operates using 16x oversampling methodology.

Each UART bit duration is divided into 16 sampling intervals. The receiver validates and samples the incoming serial bit at the center of the bit period.

Center sampling improves communication reliability by minimizing timing errors and noise sensitivity.

The `sample_count` counter tracks oversampling intervals and performs data sampling when `sample_count` equals 7.

Timing Behaviour

The UART transmitter and receiver operate synchronously with their respective baud clocks.

Transmission Timing

- Input data is loaded during IDLE state.
- Start bit is transmitted first.
- Data bits are transmitted serially using baud clock.
- Stop bit is transmitted after completion of data bits.
- `xmit_doneH` is asserted after transmission completion.

Reception Timing

- Receiver continuously monitors UART line.
- Falling edge detection indicates start bit.
- Start bit validation is performed using oversampling.
- Data bits are sampled at center position.
- Stop bit validation completes reception.

- rec_readyH indicates successful data reception.
-

Testbench Architecture

The verification environment consists of:

- Top Testbench Module
- Driver Module
- Scoreboard Module
- Reference Model
- DUT Instantiation

The driver module generates UART transmission stimulus and applies multiple test vectors to the DUT.

The scoreboard compares DUT output data with reference model outputs and displays PASS or FAIL messages based on comparison results.

The reference model independently models UART transmitter and receiver behavior to generate expected outputs.

The DUT and reference model operate simultaneously using identical input stimulus.

Driver Functionality

The driver module generates UART transmission requests using reusable tasks.

The transmit task:

- Waits until transmission becomes idle
- Applies transmit data
- Generates xmitH pulse
- Synchronizes with baud clock

Multiple test vectors are applied including:

- 8'hAA
- 8'hFF
- 8'h00
- 8'hD3
- 8'h33

- 8'h55

These patterns help verify:

- Alternating bit transmission
 - All zeros condition
 - All ones condition
 - Random data reception
-

Scoreboard Functionality

The scoreboard module compares DUT output data against the reference model output.

If both values match:

- PASS message is displayed

Otherwise:

- FAIL message is displayed

The scoreboard also displays:

- DUT data
- Reference data
- Simulation time

This self-checking verification environment improves automation and functional validation.

Quality of Code Assessment

The RTL design and verification environment were analyzed using simulation and coverage techniques.

The design follows modular coding methodology with separate implementation for:

- Transmitter
- Receiver
- Baud generators
- Verification environment

Coverage analysis was performed using QuestaSim.

The toggle coverage achieved for the UART design is approximately 98%, indicating that most design signals and functional paths were successfully exercised during simulation.

The high toggle coverage confirms that the applied testcases effectively validated UART transmission and reception functionality.

Results

The simulation results confirm successful UART communication between transmitter and receiver modules.

The transmitter successfully converted parallel input data into serial UART frames consisting of start bit, data bits, and stop bit.

The receiver correctly reconstructed the transmitted serial data using:

- 16x oversampling
- Center sampling methodology
- Start bit validation
- Stop bit verification

All major testcases passed successfully during simulation.

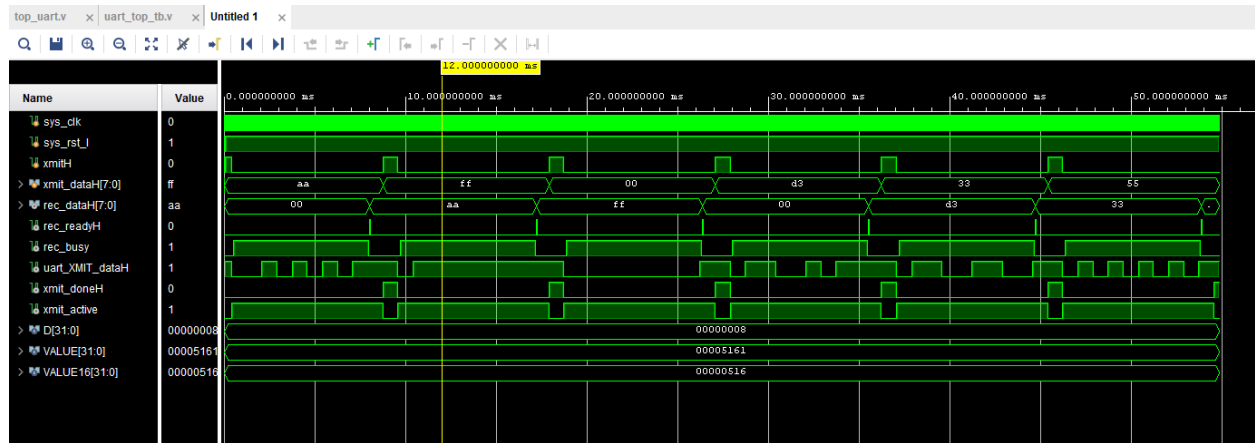
The receiver accurately received multiple input data patterns including alternating patterns, all zeros, all ones, and random values.

The verification environment successfully compared DUT outputs against the reference model outputs.

The project was simulated using:

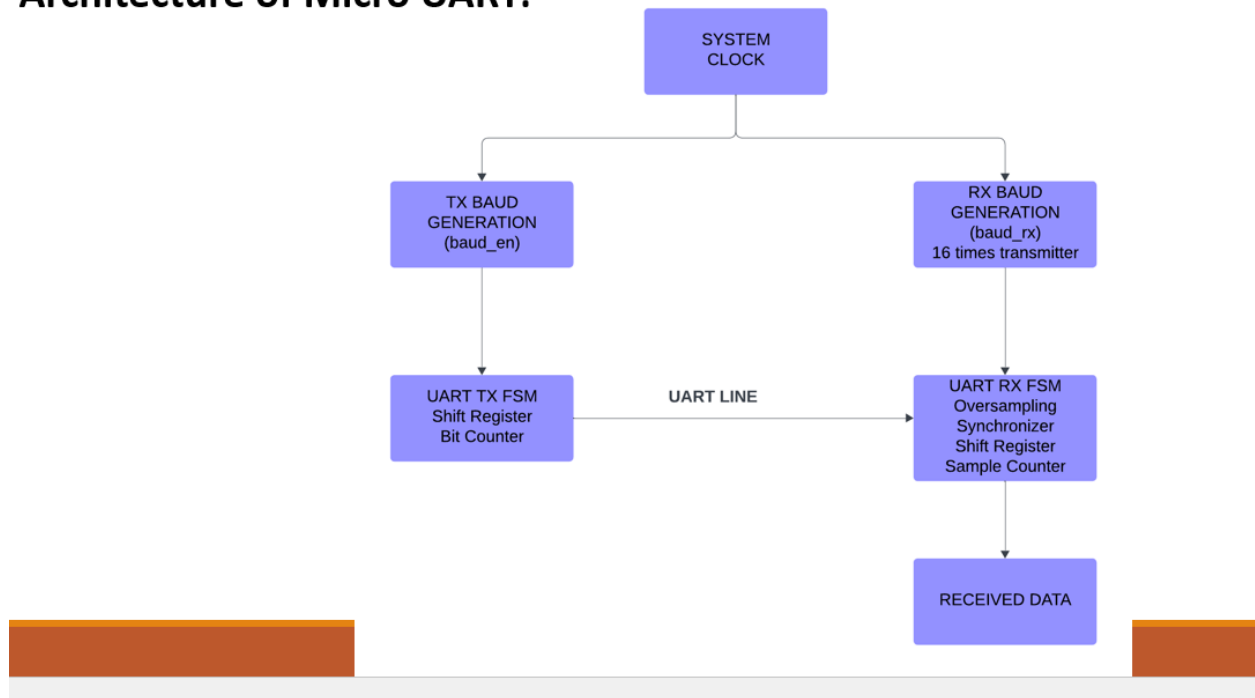
- Vivado for RTL simulation
 - QuestaSim for coverage analysis and waveform verification
-

Waveforms



Block Diagram

Architecture of Micro UART:

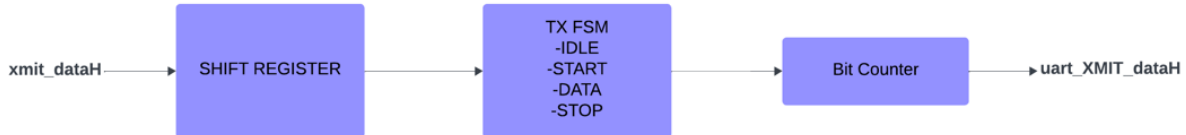


Expanded Block Diagram

Flow of UART:



Transmitter Flow:



Receiver Flow:



Coverage Report

localhost:5995 (feserver.mirafr.com:95 (krishnagirish)) - RealVNC Viewer

Applications: Questa Coverage Report, covReport - File Manager, Terminal - krishnagirish...

File://home/krishnagirish/Desktop/test/covReport/pages/_frametop.htm 80%

Centos Wiki Documentation Forums

Questa Design Coverage

Scope: /uart_top_tb/DUT

Instance Path: uart_top_tb/DUT

Design Unit Name: work_top_uart

Language: Verilog

Source File: top_uart_tb.v

Coverage Summary By Instance:

Scope	TOTAL	Statement	Branch	Toggle	FSM State	FSM Trans
TOTAL	95.34	96.87	92.85	100.00	100.00	83.33
DUT	100.00	-	-	100.00	-	-
hl	100.00	100.00	100.00	100.00	-	-
hl	94.42	95.55	90.47	100.00	100.00	83.33
hl	95.86	97.67	94.11	100.00	100.00	83.33

Local Instance Coverage Details:

Coverage Type	Blas	Hits	Misses	Weight	% Hit	Coverage
Topdown	54	54	0	1	100.00%	100.00%

Recursive Hierarchical Coverage Details:

Coverage Type	Blas	Hits	Misses	Weight	% Hit	Coverage
Statements	96	93	3	1	96.87%	96.87%
Branches	42	39	3	1	92.85%	92.85%
Toggles	174	174	0	1	100.00%	100.00%
FSMs	20	18	2	1	90.00%	91.66%
States	8	8	0	1	100.00%	100.00%
Transitions	12	10	2	1	83.33%	83.33%

22:27 ENG

The toggle coverage analysis was performed using QuestaSim.

The UART design achieved approximately 95.34% coverage during simulation. The high toggle coverage indicates that most internal signals, states, and functional paths were exercised successfully.

The applied testcases verified UART transmission, reception, oversampling logic, baud generation, FSM transitions, shift register operations, and control signal activity.

Coverage Observation

- UART transmission functionality was successfully verified.
 - Receiver oversampling and center sampling logic were exercised.
 - FSM state transitions for transmitter and receiver were covered.
 - Multiple data patterns were validated successfully.
 - Most internal design signals achieved toggle activity during simulation.
-

Conclusion

This project successfully implemented and verified a parameterized Micro UART using Verilog HDL.

The UART design supports asynchronous serial communication using start and stop bits with configurable data widths from 6-bit to 8-bit.

The transmitter and receiver modules were implemented using FSM-based architecture. The receiver successfully utilized 16x oversampling and center sampling techniques for reliable serial data reception.

A self-checking verification environment consisting of driver, scoreboard, and reference model was developed to validate UART functionality.

Simulation and coverage analysis confirmed successful UART transmission and reception for multiple data patterns.

The project provided complete understanding of:

- UART protocol implementation
- FSM design
- Oversampling techniques
- Serial communication methodology
- RTL verification flow

- Self-checking testbench development
 - Coverage analysis
-

Future Work

Several enhancements can be implemented in the future to further improve the UART design and verification environment.

Possible future improvements include:

- Addition of parity bit support
- Configurable stop bit selection
- FIFO-based buffering support
- Error detection mechanisms
- Support for higher baud rates
- UVM-based verification environment
- FPGA implementation and hardware validation
- Functional coverage implementation